Microservice Communication Styles

sync when you need the answer now

Watch the 4-min video

In one line: call synchronously (REST/gRPC) only when the caller needs the answer to continue, and protect that call with timeouts and circuit breakers; send everything else asynchronously through a broker, so services are decoupled in time. Decide per interaction, not per system.

1 One slow call breaks the chain

Checkout calls payment, which calls fraud, which calls scoring. Scoring gets slow, every caller's threads fill up waiting, and checkout times out. Yet some calls really do need an answer right now: you can't confirm an order without knowing the charge succeeded.

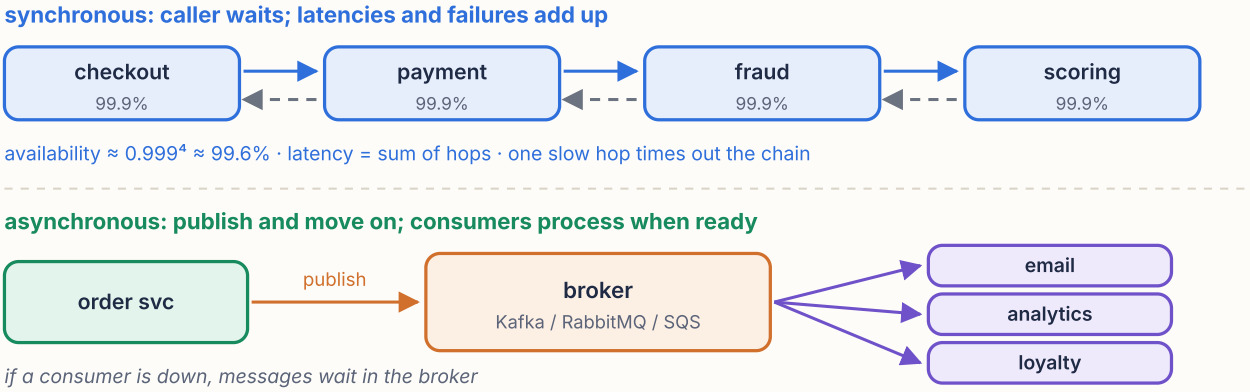


Figure 1. A synchronous chain multiplies availability and adds latency; a broker decouples sender and receivers in time.

2 Synchronous: request and response

The caller sends a request over **REST** (HTTP/JSON) or **gRPC** (HTTP/2, Protocol Buffers) and waits. It is simple, familiar and easy to debug, and fits queries and commands whose result the caller needs. The cost is **temporal coupling**: both services must be up at the same moment.

- Availability multiplies along a chain: five services at 99.9% give $0.999^5 \approx$ **99.5%** (lesson 1).
- Latency adds up; tail latency of the slowest hop dominates.
- Failures cascade unless every call has a **timeout**, limited **retries with backoff** and a **circuit breaker** (lessons 4–6).
- Keep chains short: ideally one or two synchronous hops per user request.

3 Asynchronous: messages through a broker

The sender publishes an **event** ("OrderConfirmed", a fact) or a **command** ("SendReceipt", a request to one receiver) to a broker such as Kafka, RabbitMQ, Amazon SQS/SNS or Azure Service Bus, then moves on. Receivers process when ready; if one is down, messages wait. This removes temporal coupling, absorbs spikes, and lets new consumers subscribe without changing the sender.

Patterns: in **choreography** services react to each other's events with no coordinator; in **orchestration** one service sends commands and tracks the steps (lesson 34). Request/response can even run over messaging with a **reply queue** and a **correlation ID**. The costs: eventual consistency, duplicate delivery (consumers must be idempotent), and harder debugging, so you need correlation IDs and tracing (lesson 36).

	Synchronous	Asynchronous
Coupling in time	both must be up	receiver can be down; messages wait
Result	immediate	later; eventual consistency
Load spikes	passed straight through	absorbed by the queue
Adding a consumer	caller must change	just subscribe
Debugging	simple call stack	needs correlation IDs and tracing

4 Choose per interaction

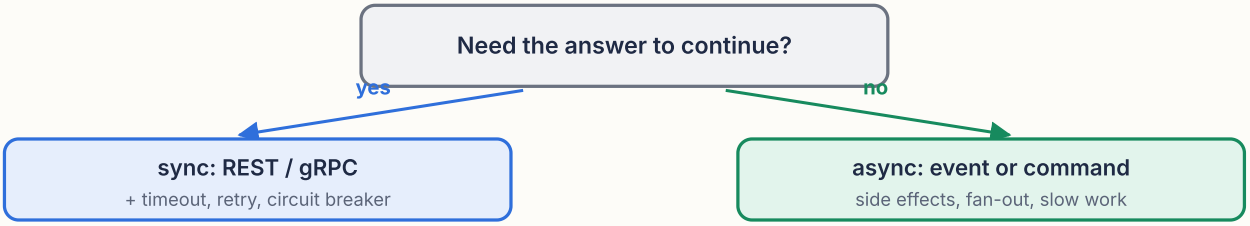


Figure 2. The key question. Most real systems mix both styles.

WORKED EXAMPLE: CHECKOUT AND RECEIPTS

The order service calls payment **synchronously** (gRPC, 2-second timeout, one retry with an idempotency key) because it must know whether the charge succeeded before confirming. Then it publishes `OrderConfirmed`. The email service consumes it **asynchronously** and sends the receipt; analytics and loyalty subscribe to the same event. If email is down for ten minutes, receipts are ten minutes late and checkout keeps working.

COMMON MISTAKES

- Long synchronous chains on the user's critical path.
- Sync calls with no timeout, so one slow service hangs everyone.
- Assuming exactly-once delivery: consumers must handle duplicates.
- Making everything async, even when the user is waiting for the answer.

5 Key terms

Temporal coupling: both sides must be available at the same time

Choreography: services react to events, no coordinator

Correlation ID: ID carried through a flow to link its messages

Event vs command: a fact for anyone vs a request to one receiver

Orchestration: one service directs the steps

Idempotent consumer: processing a message twice has no extra effect

6 Interview questions

Q1. What is temporal coupling? BASIC

Both caller and callee must be up at the same moment, as in a synchronous call; async messaging removes it because messages wait in the broker.

Q2. What is the difference between an event and a command? BASIC

An event states a fact that happened, for any number of subscribers; a command asks one specific receiver to do something.

Q3. When should services talk synchronously vs asynchronously? SCENARIO

Sync when the caller needs the answer now to continue, like authorising a payment, protected by timeouts and circuit breakers; async for side effects, fan-out and slow work.

Q4. Five services each at 99.9% are called in a chain. What's the availability? SCENARIO

About $0.999^5 \approx 99.5\%$, before counting the caller itself; make the non-essential calls async or optional to raise it.

Q5. Can you do request/response over a message broker? TRICKY

Yes: send the request with a reply-to queue and a correlation ID, and match the reply; it buffers load but the caller still waits for the result.

Q6. Why must async consumers be idempotent? TRICKY

Brokers usually deliver at least once, so redeliveries after crashes or timeouts happen; the consumer must detect or tolerate duplicates.

Next: SD-L-007 · API Gateway Pattern backendroadmap.com